

Seamless End-to-End Containerized HPC Environments

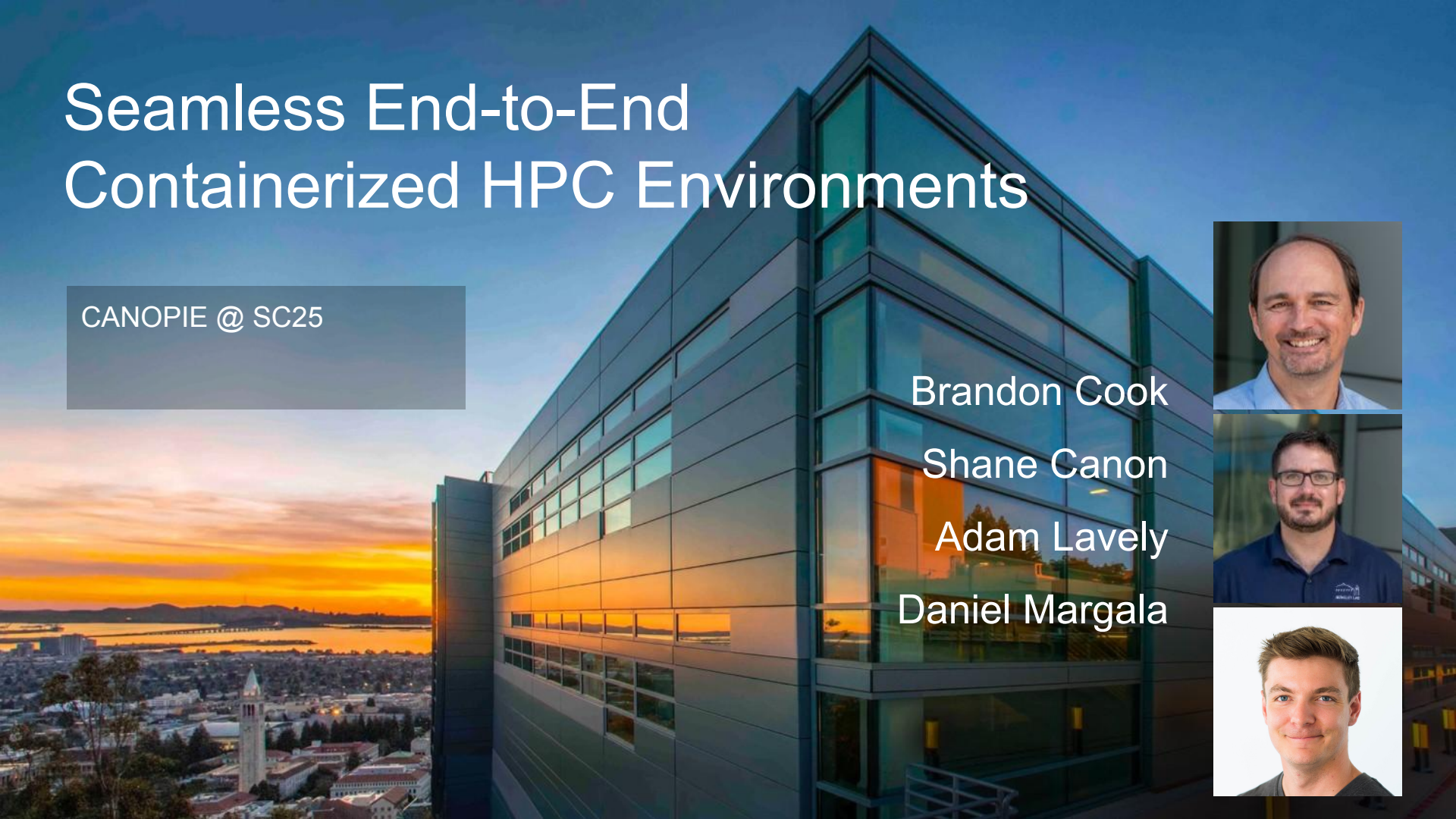
CANOPIE @ SC25

Brandon Cook

Shane Canon

Adam Lavelly

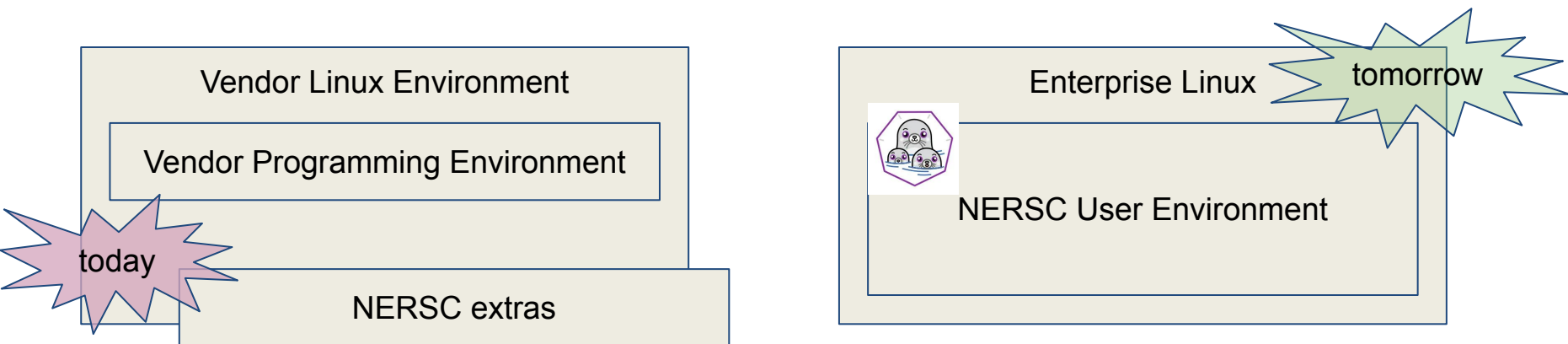
Daniel Margala



An HPC user environment is...

The *overall* structure within which a user operates:

shells, toolchains, libraries, tools through modules on a Linux OS with network, filesystem, gpu, cpu, ...

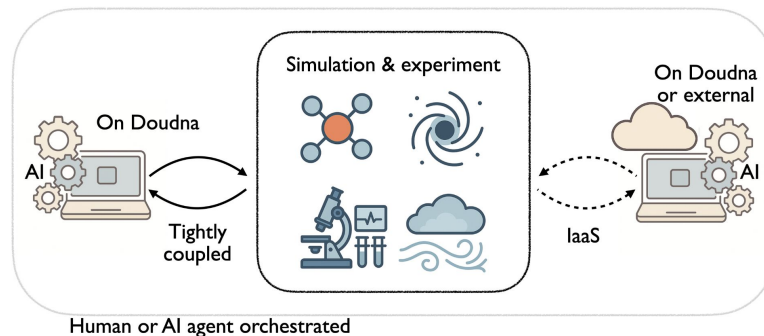
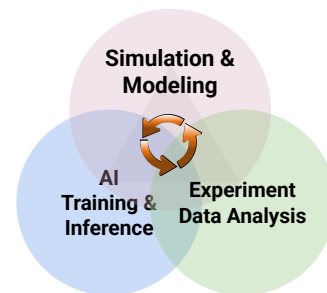


The traditional approach is monolithic

- Monolithic
- One OS, often customized by system vendor
- One Environment, *tightly coupled* to the hardware and OS
- Major updates to “core” ~annually
- Typical deployments include baked into images or shared file systems
 - On Perlmutter we have squashfs files on NFS with loopback mounts
 - i.e. uniform on a system

Challenges with traditional HPC environments

1. Growing complexity of scientific workflows
2. Multiple interfaces
3. Reproducibility vs flexibility



When? What?

We have a campaign to run!

Please *change nothing*

I *need* that patch released *yesterday!*

Notice:
The CVEs will continue until morale improves.

Upgrades tomorrow!

For our new runtime I need *full control over everything*

We want simple common defaults for our collaboration!



Containers to the rescue, but how?

1. Design principles
2. Representative workflows
3. Technical approach



Guiding design principles

User Experience

Transparent
by default

Complexity is
opt-in

Consistency

Technical Longevity

Open Source

Composition

Standards

Modern DevOps

CI/CD

IaC

Secure

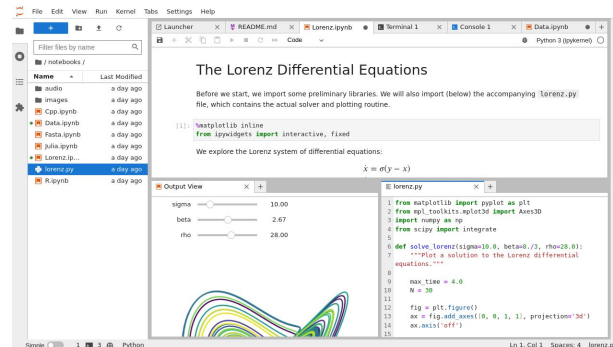
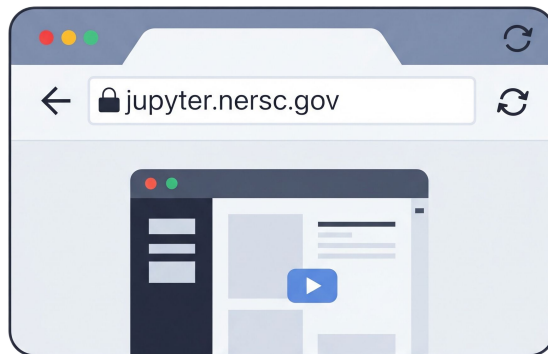
Two core representative workflows



```
laptop>  
ssh doudna.nersc.gov
```



```
doudna>  
mpif90 science.f90  
sbatch job.sh
```



NERSC Landscape

Community
support, training, documentation, ...

product management (requirements, release cycle, roadmapping...)

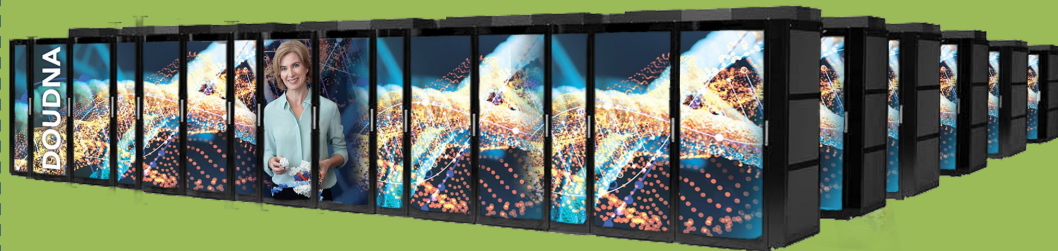
monitoring and
analytics

Upstream
ecosystem /
model /
language /
component
R&D

User defined
environments

**Environment
Definition
and builds**

System implementation



security

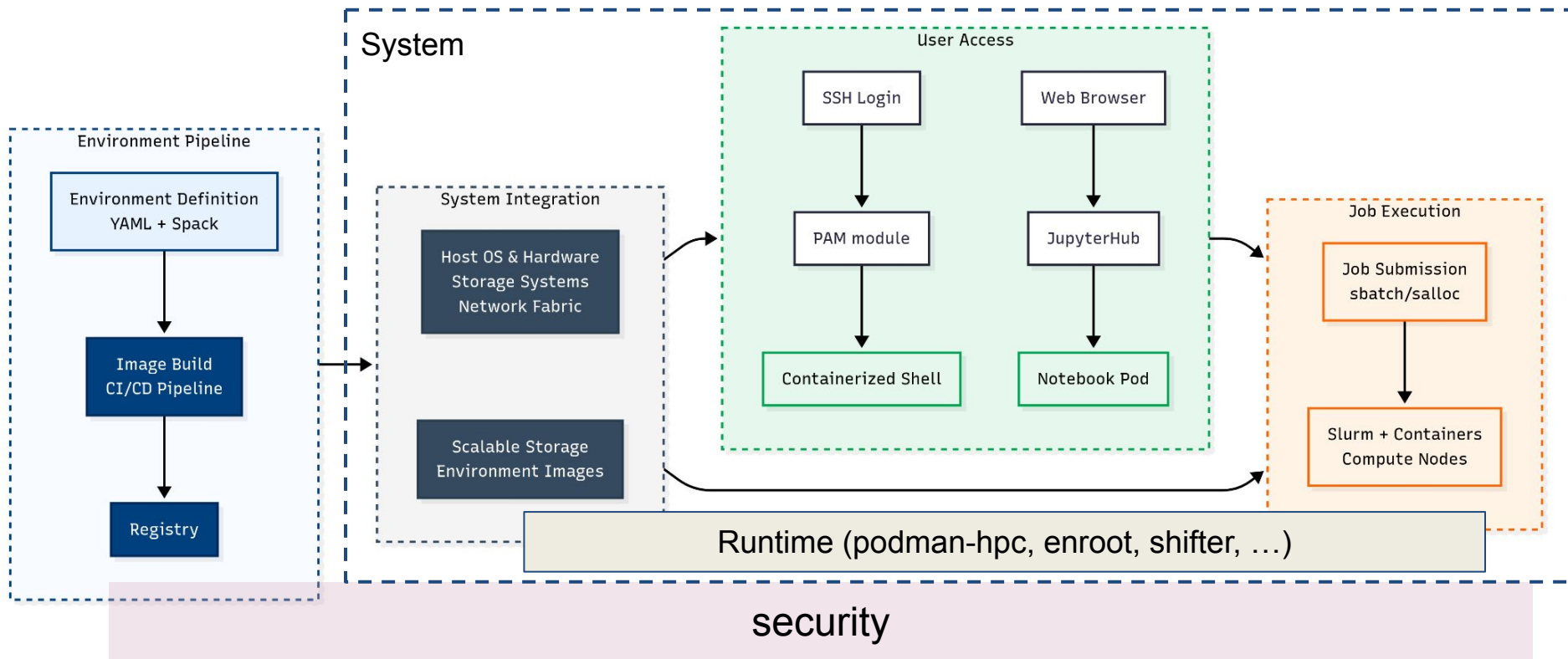
portability to N9, TACC, *LCF, Cloud, ...

staff provided applications

“classic” HPC... in containers!

- emphasis on the CLI workflow for today
- Everything in the image is publicly available!
- “login” remains emulated (for general use)
- preview available on Perlmutter!

Technical approach

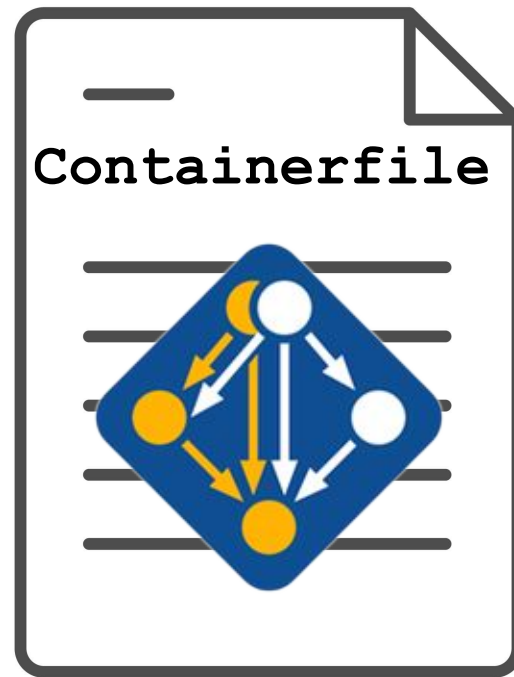


Environment specification and construction

Essential components

- OCI compliant **Containerfile**
- A spack environment **spack.yaml**

1. A brief tour of these files
2. Facility workflows
3. User workflows



OCI compliant Containerfile

```
FROM nvcr.io/nvidia/cuda:12.9.1-devel-ubuntu24.04
```

```
RUN apt-get install <os_packages>
```

```
COPY spack.yaml /opt/spack
```

```
RUN $SPACK -e /opt/spack install
```

```
RUN $SPACK -e /opt/spack module tcl refresh
```

```
RUN pip install jupyterlab jupyterhub
```

```
# init lmod; populate /etc/profile.d
```

Spack environment `spack.yaml`

A suite of “core” HPC packages covering compilers, communication, I/O, math libraries, environment management and build tools, etc

`specs:`

- `openmpi@5 schedulers=slurm fabrics=ofi,cma`
- `openblas threads=openmp`
- `hdf5 +fortran+threadsafe+hl`
- `papi`
- `miniconda3`
- ...

`mpileaks@1.2:1.4 +debug ~qt target=x86_64_v3 %gcc@15 ^libelf@1.1 %clang@20`



Environment definition is amenable to modern highly automated software development workflows



gitlab.nersc.gov

source control
issues
merge requests
tags



registry.nersc.gov

hosting
scanning



n10-dev.nersc.gov

CI/CD
ReFrame tests

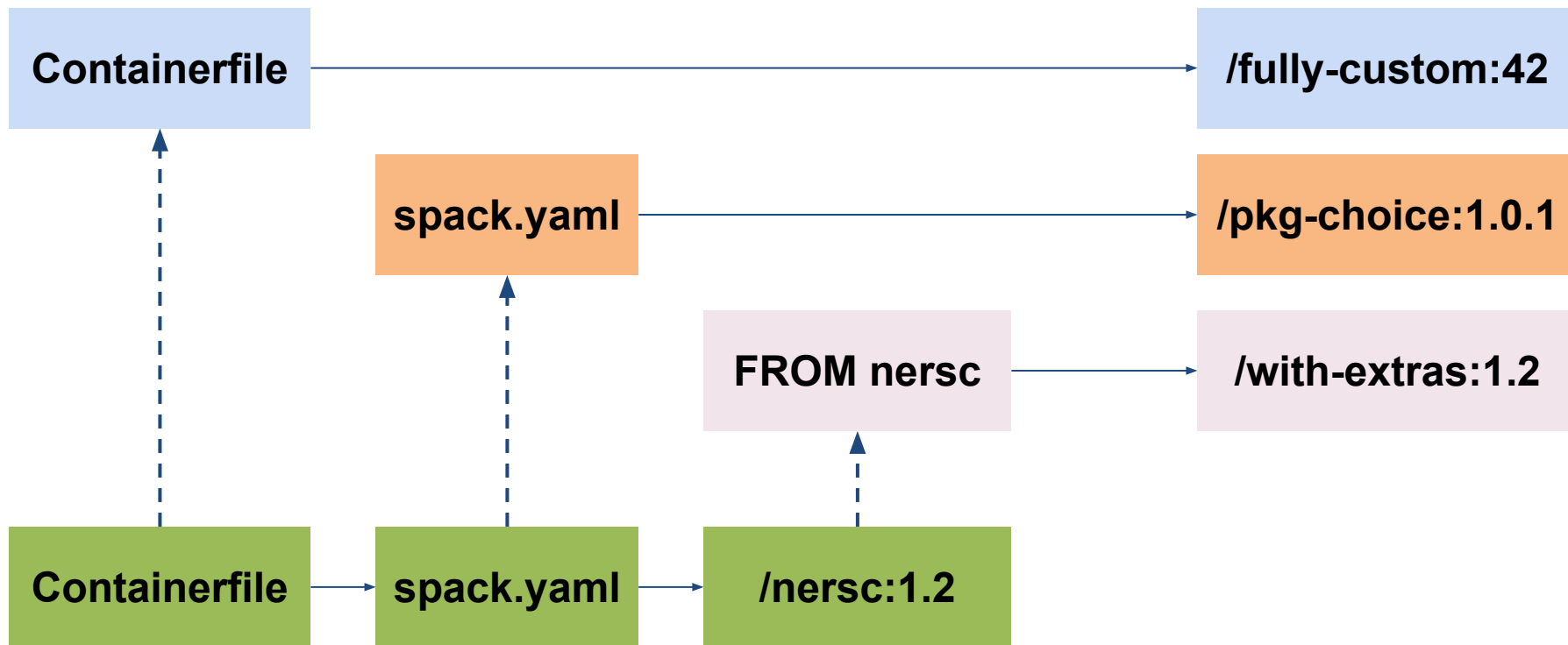


doudna.nersc.gov



to users!

Users have *options* to customize!

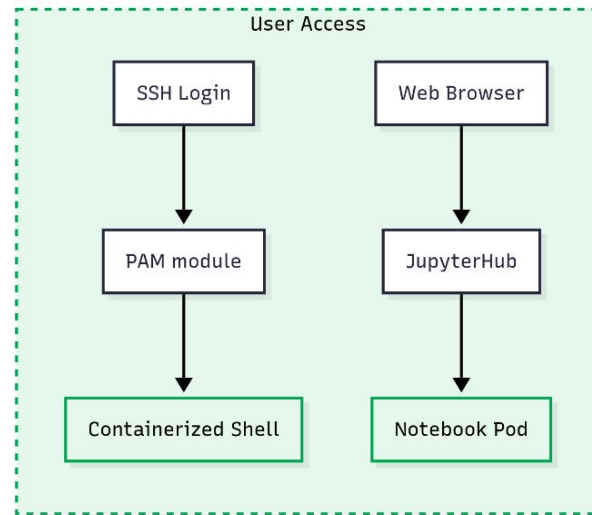


Session provisioning

How can users get a `$SHELL` on the system?

- *in* a container
- “same” UX i.e. `ssh $system`

1. Survey
2. PAM module prototype



For jupyter, `z2jh` provides a straightforward solution.

Existing options did not meet our design criteria

Approach	Pro	Con
podmansh	Redhat support	inflexible, deployment model
ContainerSSH	User isolation	not OpenSSH, ephemeral
Cray/HPE UAI	Vendor integration	Vendor lock, adoption and roadmap

A variety of models are possible



k8s-based

custom session manager service

- k8s networking with Slurm CLI
- complex
- reuse in converged platforms

Jump Host

minimal “baremetal” access host

- extra steps for users
- not “seamless”

PAM module

hook into existing sshd flow

- small delta for administration
- small surface area (i.e. not k8s)
- still custom code

Pluggable Authentication Modules (man 8 pam)

```
> OpenSSH: /etc/pam.d/sshd  
  
session required pam_container.so
```

```
> pam_container.so  
  
pam_sm_open_session() {  
    pid = create_session()  
    setns(pidfd_open(pid))  
}
```

```
create_session {  
    c = podman run <image> catatonit -P  
    return pid( c )  
}
```

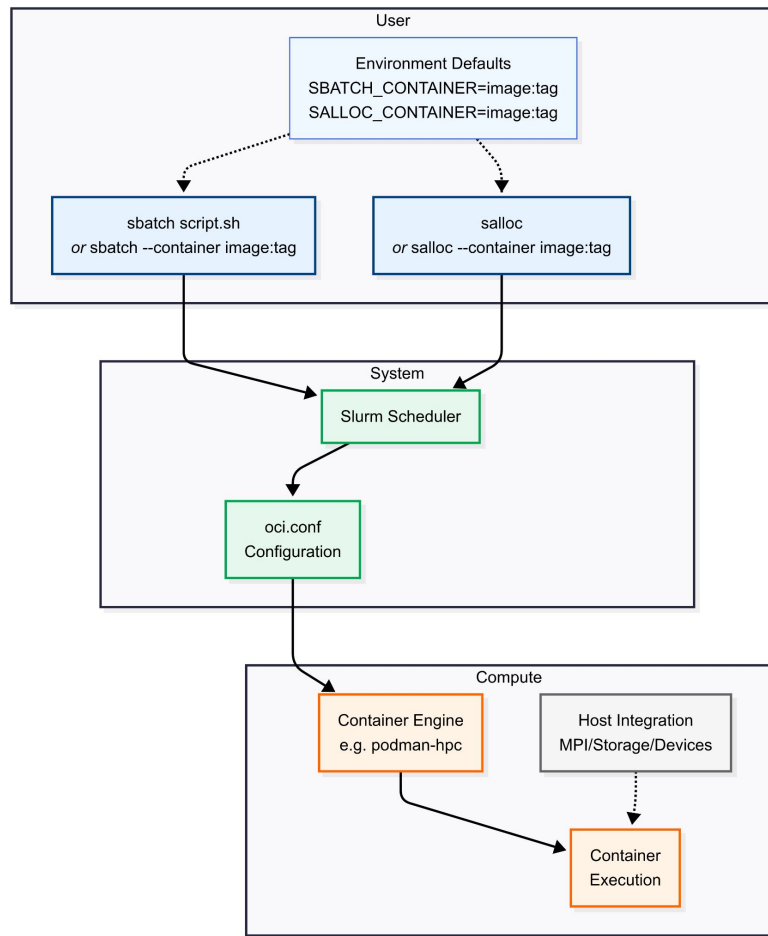
```
> user session container  
1 catatonit -P  
2 bash --login
```



Scheduler integration is essential for *seamlessness*

Job environment must match submission environment by default

Many plugin options on the market, native Slurm support since 21.08



native Slurm integration

```
SBATCH_CONTAINER=<image> sbatch ...
```

==

```
sbatch --container <image> ...
```

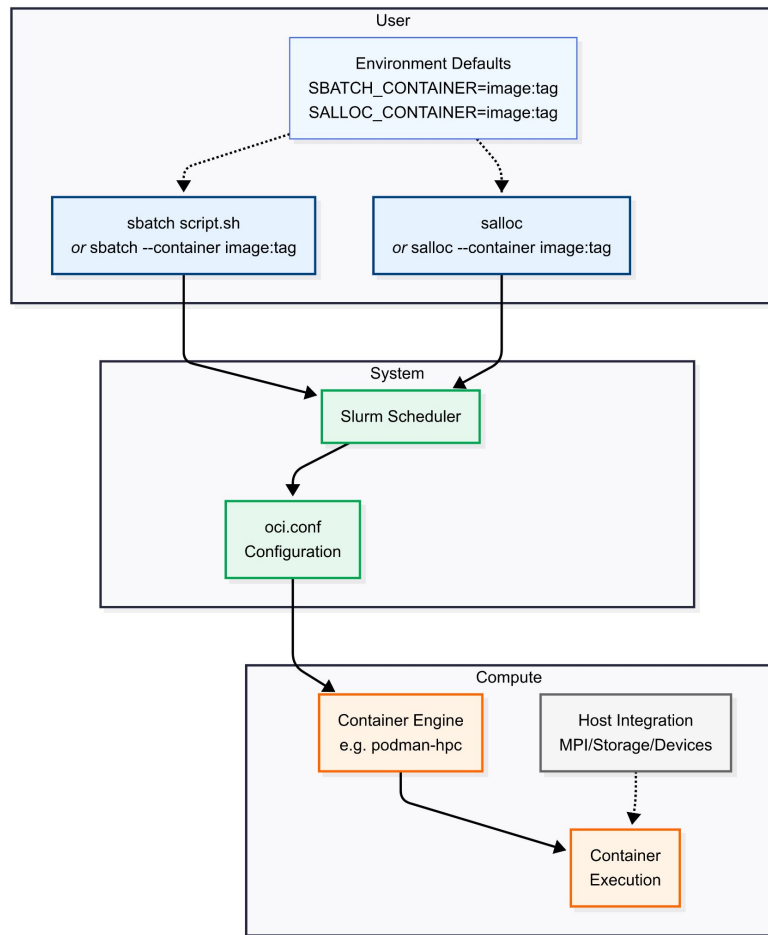
```
> oci.conf
```

```
RuntimeRun="/path/to/run <args> %@"
```

```
> run
```

```
IMAGE=$1; ...; shift 9
```

```
<engine> run <opts> <image> $@
```



System integration is multifaceted

Slurm CLI packaging is largest current pain point.

- needs to be outside of the image
- sprawling dependencies
- *completely coupled* slurmd

Converged infrastructure

bind mounts → CDI, CSI, ...

Scalable storage and launch

usersns keep-id, image cache, shared-run, ...

CLI Tools

api.nersc.gov → python → pip install nersc-cli

MPI?

d) all of the above!

(default)

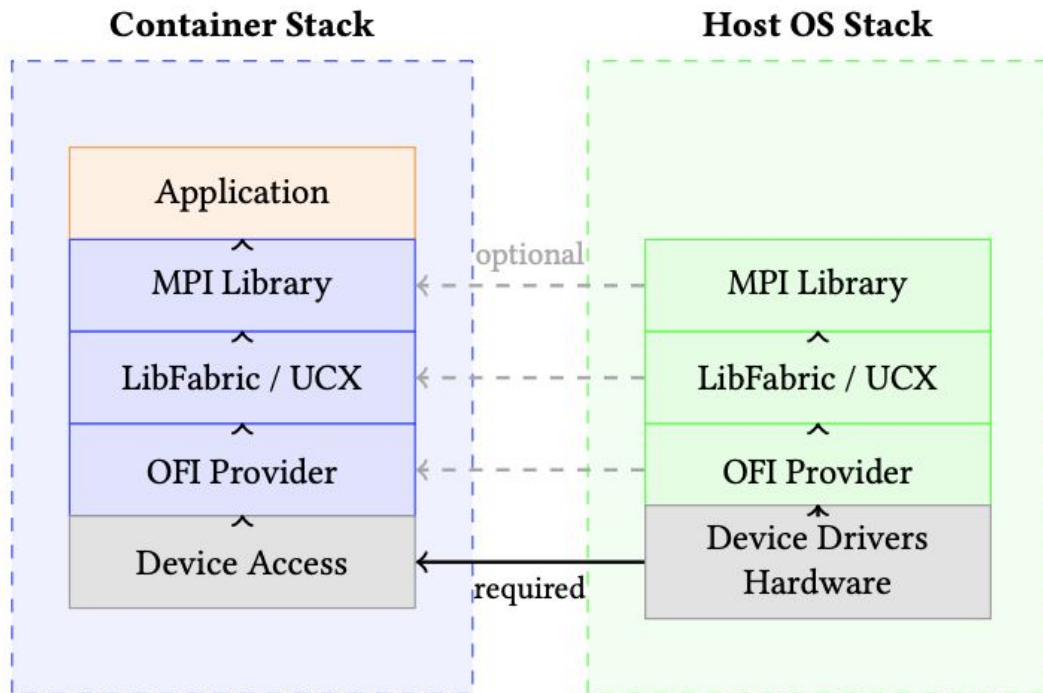
- CDI + MPI/ NCCL library in image

Bring your own container

- MPI ABI compatible optimized MPI in Host OS

Alternative models?

- CDI and low-level comms (UCX, OFI) available!



Questions?

seamless end-to-end
containerized
environments

Acknowledgements

Chris Samuel and NERSC/CSG for help with Slurm, Justin Cook for Spack, Rahul Gayatri, Paul Lin and the whole N10 1.05 team, ... and many more who enable the whole ecosystem.

